

School Timetabling -- A Case in Large Binary Integer Linear Programming

Author(s): Arabinda Tripathy

Source: *Management Science*, Dec., 1984, Vol. 30, No. 12 (Dec., 1984), pp. 1473-1489

Published by: INFORMS

Stable URL: <https://www.jstor.org/stable/2631473>

REFERENCES

Linked references are available on JSTOR for this article:

https://www.jstor.org/stable/2631473?seq=1&cid=pdf-reference#references_tab_contents

You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Management Science*

JSTOR

SCHOOL TIMETABLING—A CASE IN LARGE BINARY INTEGER LINEAR PROGRAMMING*

ARABINDA TRIPATHY

Indian Institute of Management, Vastrapur, Ahmedabad 380015, India

A timetabling problem is formulated as a large integer linear programming problem in 0-1 variables. A solution method based on Lagrangean relaxation coupled with subgradient optimization is presented. The solution method also incorporates a branch and bound procedure which takes advantage of special ordered sets of variables. We present computational results for a large timetabling problem involving 900 subjects in a one-year graduate program.

(PROGRAMMING—INTEGER; APPLICATIONS; LAGRANGEAN RELAXATION)

1. Introduction

The timetabling problem has been of interest since late 1950's. Various researchers have dealt with the problem in different ways. The problem is of interest even now, largely due to its varied nature, complexity and last but not the least its large size.

In simple words, the timetabling problem can be defined as the scheduling of a certain number of meetings, which are to be attended by a specific group of students and a teacher (or teachers), over a definite period of time, requiring certain resources (e.g. rooms, teaching aids etc.) in conformity with the availability of resources and fulfilling certain other requirements.

The approaches for solution of the timetabling problem have been from pure simulation of hand construction of a time table (Barracough 1965, Murphy and Sutter 1964) to graph theoretic approaches (Punter 1977a, b, Welsh and Powell 1967, and Wood 1969). The programs developed by Purdue University in 1956 and CLASS (Class Load And Student Scheduling) developed by IBM shortly thereafter fall in the category of simulation of hand construction of a timetable. Most of the previous studies have dealt with school timetabling problems (Appleby et al. 1961, Aust 1973, 1976, Barracough 1965, Csima 1965, Dempster 1968, 1971, DeWerra 1971, 1978, Dutt Roy 1972, Gottlieb 1963, Junginger 1972, Lawrie 1969, Lawrie and Veitch 1975, Lions 1966, 1967, Local Government Report, McDiarmid 1972, Murphy and Sutter 1964, Oakford et al. 1966, Punter 1977a, b, Ulph 1977, and others). There are relatively few studies dealing with the College or University timetabling problems (Akkoyunlu 1973, Almond 1966, 1969, Anthonisse and Lageweg 1979, Britton and Farley 1971, Sharrat 1970, White 1976, Yule 1967) and some with examination timetabling problem (Laporte and Desroches 1982, Wood 1968). That comparatively less amount of work done on college/university timetabling is largely due to the opinion that it is easier to construct a college timetable than a school timetable, as the conflicts are likely to be less in a college situation.

This generalisation does hold good in some cases but certainly not in case of graduate school timetabling considered here. The present study can be extended to other university/college situations and to other similar problems. One of the characteristics of the timetabling problem is its large size. This becomes a great hinderance to solving the timetabling problem using mathematical programming techniques. In a mathematical programming formulation of the timetabling problem all the variables are integers, which is a further deterrent for using such a technique. The use of relaxation techniques based on Lagrangean multipliers to solve the Integer Linear Programming problem has shown great promise since the beginning of the 1970's. Encouraged by its success the present study attempts to solve the timetabling problem using mathematical programming techniques based on Lagrangean relaxation.

2. The Timetabling Problem

The problem considered here deals with scheduling of about 900 subjects over an academic year, in a graduate program of one academic year duration. The students are enrolled in 25 different streams (specialisation) in the graduate program. There is an increased complexity due to the fact that students from 4 to 6 different streams have to

* Accepted by Marshall L. Fisher, former Departmental Editor; received June 30, 1982. This paper has been with the author 5 months for 2 revisions.

study a particular subject. When such a subject is scheduled in a period, all the other subjects which are also to be studied by the students from these 4 to 6 different streams cannot be scheduled in the same period. Such situations are quite common in the timetabling problem considered here. The academic year is divided into three terms. Some of the subjects are spread over more than one term, whereas the others are for one term only. The timetable is made separately for all the three terms, except that the continuing subjects are scheduled in the same periods as in the previous term as far as possible. The term timetable has to deal with about 400 to 450 subjects. If the week is of 30 periods, a direct mathematical programming formulation results in over 12000 variables and nearly as many constraints. All the variables are integer valued. The size of the integer linear programming problem so formulated becomes unmanageable to handle. In order to reduce the size of the problem the variables need to be redefined, and this is achieved through a 'grouping' operation.

3. Grouping Operation

The most difficult condition to satisfy in a timetabling problem is that no student is expected to attend more than one subject in a period. If this condition is not satisfied the problem is infeasible. Student groups are formed with this condition in view. In general a student group is the group of students who are enrolled in the same stream. When a student group is defined in the above manner, all the subjects which the group is supposed to attend (though a single student may not be required to attend all the subjects) are to be scheduled in different periods, i.e. they cannot be scheduled in parallel or concurrently. In other words, these subjects become conflicting among themselves. Let $S_1, S_2, S_3, \dots, S_n$ be the subjects to be attended by a particular student group. $p_1, p_2, p_3, \dots, p_n$ are the number of periods per week required by the subjects $S_1, S_2, \dots, S_n$ respectively. If $\sum_{i=1}^n p_i > NP$ (NP = total number of periods per week), then the timetabling problem is infeasible.

This infeasibility can be resolved by re-forming the student groups. The actual nature of the problem stated above is as follows:

The particular student group (enrolled in a particular stream) has some compulsory subjects, and has to take some optional subjects from a few other subjects. Say, S_1 and S_2 are compulsory subjects and any two optional subjects from $S_3, S_4, \dots, S_n$. Now the student group can be split into two or more student groups based on the actual options taken by the students (or on the basis of likely options to be taken by the students as in the past), e.g.

$$\begin{array}{ll} \text{student group 1} & TG1 - S_1, S_2, S_3, S_6, S_8, \dots, \\ \text{student group 2} & TG2 - S_1, S_2, S_4, S_6, S_7, S_8, \dots, \end{array} \quad \text{such that}$$

$$\sum_{i \in \text{student group}} p_i \leq NP$$

for all the student groups.

The closer the $\sum p_i$ to NP , the greater the chance of getting an infeasible solution to the ILP problem. This is because the subject S_i taken by a student group $TG1$ may also be taken by another student group $TG2$ and this increases the number of subjects in conflict with S_i and thereby makes the scheduling of S_i difficult or impossible, in the extreme case resulting in there being no feasible solution. This situation is caused by the fact that the subjects taken by a student group are greater in number than the subjects taken by a particular student of the student group. But all the subjects for a student group are in conflict among themselves. This is avoided by splitting the student group, whereby the number of subjects for any one student group (after splitting) is reduced, and the number of subjects in conflict with a particular subject is

TABLE 1

Student Group Subjects	A	B	C
1	X		X
2		X	
3	X		
4		X	X
5	X	X	X
6	X	X	
7		X	
8	X		X
9	X	X	X
10	X		

X indicates the subject being attended by the student group.

reduced, thereby improving the possibility of getting a feasible solution. Therefore students enrolled in a particular stream may have to be formed into a number of student groups from the feasibility point of view, such that $\sum p_i$ for a particular student group is less (or sufficiently less) than NP.

Once the students have been grouped into student groups, the next natural thing is to group the subjects into subject groups. The formation of student groups forms the basis for the formation of subject groups. A subject group is defined as the group of subjects which are attended by the same set of student groups. The procedure for the formation of subject groups is explained through an example. The relationship between the student groups and the subjects is presented in Table 1. First, the subjects which are attended by one student group only are selected, and among these, the subjects which are attended by the same student group are grouped into one subject group. In the example of Table 1, subjects 2, 3, 7 and 10 are attended by one student group only. These can be grouped as follows:

Subject group I—subjects 2 and 7, attended by student group 'B'

Subject group II—subjects 3 and 10, attended by student group 'A'.

Next the subjects which are attended by two student groups are selected and these are grouped into subject groups, by grouping the subjects which are attended by the same set of student groups. In Table 1, subjects 1, 4, 6 and 8 are attended by two student groups and these can be grouped as follows:

Subject group III—subjects 1 and 8, attended by student groups A and C.

Subject group IV—subject 4, attended by student groups B and C.

Subject group V—subject 6, attended by student groups A and B.

Next, the subjects which are attended by three student groups, then the subjects which are attended by four student groups, and so on, are considered progressively, until there are no subjects left to be considered. At this stage the formation of subject groups is complete, and every subject belongs to one and only one subject group.

The advantage of such a grouping of subjects is not apparent from the small example discussed here. But in the case of the problem dealt with in the study, nearly 400 subjects have been grouped into less than 100 subject groups, thereby reducing the number of variables to one fourth of the number of variables in the direct ILP formulation.

The other grouping which has to be done for the timetabling problem is the grouping of rooms. The formation of the room groups is based on the capacities of the rooms. The rooms of identical capacity are grouped into one room group. In the present study the rooms have been grouped into five room groups based on their capacities. A room group is associated with each subject group. This specifies the size

TABLE 2

Student Group Subject Group		A	B	C
	1		X	
	2	X		
	3	X		X
	4		X	X
	5	X	X	
	6	X	X	X

X represents that the subject group is attended by the student group.

TABLE 3

Subject Group	1	2	3	4	5	6
1				X	X	X
2			X		X	X
3		X		X	X	X
4	X		X		X	X
5	X	X	X	X		X
6	X	X	X	X	X	

X represents the conflict.

of the room required by the subject group. As the student groups who attend a particular subject group are known so the total number of students are known and hence on this basis the room group for a subject group can be specified.

The association of a room group with a subject group plays an important role in the solution strategy of the problem. Theoretically speaking, any room group whose capacity is more than the number of students attending the subject group can be assigned to the subject group. However, it is desirable to accommodate a group of students in a room whose capacity is very near to that of the strength of the group. E.g. a group of 15 students should be taught in a room suitable for 10–15 students and not in a lecture theatre capable of accommodating 100 students. In the formulation of the problem here a particular room group is associated with a subject group which is not far from reality, thus aiding the problem solution.

The subject groups, like the subjects, which are attended by a student group, are in conflict with each other and the conflicting subject groups cannot be scheduled in parallel, i.e., in the same period. A ‘conflict matrix’ is drawn to represent the conflicts between the various subject groups. A small example is presented in Table 2 (derived from Table 1) to show the construction of the conflict matrix.

It can be seen from the entry in Table 2 for the student group A that the subject groups 2, 3, 5 and 6 are in conflict among themselves. Similar other conflicts are observed from the other entries in Table 2. The entire information about conflict can be presented in a ‘conflict matrix’ as shown in Table 3.

4. Problem Formulation

The process of producing a timetable has three main phases.

- (i) Deciding the group of students who are to attend a particular subject.
- (ii) Specifying the facilities required by the subject.
- (iii) Determining when each subject is to take place.

The first two phases involve certain major decisions to be taken by the academic staff and the administrator. These often do not follow any specific mathematical rules

and hence are done manually. The first two phases when completed define completely the subjects, and these form the data for use in the third phase. The grouping operation described in the preceding paragraphs forms the first stage of this third phase.

Let

NSG = total number of subject groups,

NTG = total number of student groups,

NRG = total number of room groups,

NP = total number of periods in a week,

$$x_{ij} = 1 \quad \text{if subject group } i (= 1, 2, \dots, \text{NSG}) \text{ is scheduled in period} \\ j (= 1, 2, \dots, \text{NP}), \\ = 0, \quad \text{otherwise.}$$

Then the timetabling problem is

$$\text{maximise} \quad \sum_{i=1}^{\text{NSG}} \sum_{j=1}^{\text{NP}} c_{ij} x_{ij}, \quad \text{subject to} \quad (\text{P})$$

$$\sum_{j=1}^{\text{NP}} x_{ij} = \text{NSPG}_i; \quad i = 1, 2, \dots, \text{NSG}, \quad (\text{P-I})$$

$$\sum_{i \in R_k} x_{ij} \leq a_{kj}; \quad j = 1, 2, \dots, \text{NP}, \quad k = 1, 2, \dots, \text{NRG}, \quad (\text{P-II})$$

$$\sum_{i \in T_l} x_{ij} \leq 1; \quad j = 1, 2, \dots, \text{NP}, \quad l = 1, 2, \dots, \text{NTG}, \quad \text{where} \quad (\text{P-III})$$

c_{ij} —Objective function related to the desirability of x_{ij} , e.g. if it is desired to schedule subject 'i' to period 'j' for some reason, say due to the preference of an external teacher, c_{ij} can be given a high value. Conversely, for an undesirable combination of 'i' and 'j', c_{ij} can be given a low value (high value). Otherwise c_{ij} may be zero. When all the c_{ij} are zero, the problem is one of finding a feasible solution.

NSPG_i —total number of periods per week to be scheduled to subject group 'i'.

a_{kj} —number of available rooms in period 'j' of room group 'k',

R_k —subset of subject groups requiring room group 'k',

T_l —subset of subject groups attended by student group 'l',

P-I—is the constraint set related to the total number of periods required per week by the subject groups.

P-II—is the constraint set related to the availability of rooms.

P-III—is the constraint set, which takes care of the fact that a student group cannot attend more than one subject group in a particular period. This is the same as what is represented in the 'conflict matrix,' i.e. the conflicting subject groups are not to be scheduled simultaneously.

The grouping operation reduces the size of the problem considerably. However, the size of the problem still remains too large to be solved by conventional ILP solution methods. One of the obvious choices is to use relaxation techniques, due to its reported success in solving various types of problems.

5. Lagrangean Relaxation

One of the most common relaxation techniques is to relax the integrality condition on the variables and treat them as continuous variables, i.e. the ILP is relaxed into a simple linear programming problem. The integrality condition is checked after solving

the relaxed LP and, if necessary, branch and bound is resorted to satisfy the integrality conditions. Another form of relaxation is to remove a set of constraints altogether. In such a case, after the solution of the relaxed problem, the feasibility of the removed constraints is checked and the constraints restored if necessary. Yet another method is to maintain the effect of this set of constraints by 'dualising' them or giving them a 'price' and forming what is known as a Lagrangean relaxation of the original problem.

The Lagrangean Relaxation of an ILP problem can be presented in the following standard form (Geoffrion 1974).

The general integer linear programming problem is:

$$\text{maximise } cx, \quad \text{subject to } x \geq 0, \quad Ax \leq b, \quad Bx \leq d, \quad x_j \text{ integer, } j \in I. \quad (\text{IP})$$

An LP relaxation of (IP) is

$$\text{maximise } cx, \quad \text{subject to } Ax \leq b, \quad Bx \leq d, \quad x \geq 0. \quad (\bar{\text{IP}})$$

The integrality condition $x_j \text{ integer, } j \in I$, is checked after solving the problem $(\bar{\text{IP}})$ and if it is satisfied the solution of $(\bar{\text{IP}})$ is a solution of the original problem (IP).

A Lagrangean relaxation of (IP) relative to $Ax \leq b$ and conformable nonnegative vector λ is defined as

$$\begin{aligned} \omega(\lambda) = \text{maximise } & cx + \lambda(b - Ax) \quad \text{subject to} \\ & x \geq 0, \quad Bx \leq d, \quad x_j \text{ integer, } j \in I. \end{aligned} \quad (\text{IPR}_\lambda)$$

In the problem (IPR_λ) the term $\lambda(b - Ax)$ in the objective function is nonnegative, as λ is nonnegative and the $(b - Ax)$ is nonnegative, when the constraint $Ax \leq b$ is satisfied. Therefore if $\nu(\cdot)$ is used to represent the optimal solution of problem $(\cdot)$, $\nu(\text{IPR}_\lambda) \geq \nu(\text{IP})$. The vector ' λ ' in (IPR_λ) should be so selected that $\nu(\text{IPR}_\lambda)$ is as close to $\nu(\text{IP})$ as possible. Ideally the choice would be to take λ as an optimal solution to the convex program:

$$\text{minimise } \nu(\text{IPR}_\lambda), \quad \lambda \geq 0. \quad (\text{D})$$

The problem (D) is the formal Lagrangean dual of (IP) with respect to constraint $Ax \leq b$ (Geoffrion 1971). It may be noted that (D) is a convex function of λ alone. As x_{ij} is integer both in (IP) and (IPR_λ) , therefore neither (IP) nor (IPR_λ) is a concave program. As a result $\nu(\text{D})$ may not be equal to $\nu(\text{IP})$. In other words there will be a gap between the primal solution $\nu(\text{IP})$ and the dual solution $\nu(\text{D})$. This gap is referred to as the 'Duality Gap.' In a case where $\nu(\text{D}) = \nu(\text{IP})$ the duality gap is zero. The process of partitioning of the constraints to two sets $Ax \leq b$ and $Bx \leq d$ and the selection of suitable Lagrange multipliers λ are directed towards minimising the duality gap and so getting a solution as close to $\nu(\text{IP})$ as possible.

6. Subgradient Method

Various methods have been suggested for the computation of λ . The method followed in the present study is similar to that given in Fisher, Northup and Shapiro (1975) and Held, Wolfe and Crowder (1974). The function $\omega(\lambda)$ in the problem (IPR_λ) is not everywhere differentiable so an ascent method is constructed by using a generalisation of the gradient. A vector ' γ ' is called a subgradient of $\omega(\lambda)$ at $\bar{\lambda}$ if $\omega(\lambda) \geq \omega(\bar{\lambda}) + (\lambda - \bar{\lambda})\gamma$ for all λ . Any subgradient appears to point in a direction of ascent of ω at $\bar{\lambda}$. A readily available and easily computable subgradient is $\gamma = A\bar{x} - b$ where $\bar{x}$ is any solution of (IPR_λ) . The subgradient optimization method uses these subgradients to compute the multiplier vector λ . Held and Karp (1970, 1971) have also used a similar method for the travelling salesman problem.

The idea behind subgradient method is to generate a sequence (Fisher, Northup and Shapiro 1975) λ^r of nonnegative dual variables by the rule $\lambda_i^{r+1} = \max\{\lambda_i^r + \theta_r \gamma_i^r, 0\}$, $i = 1, 2, \dots, m$, where γ^r is any subgradient of ω at λ^r and θ_r is a positive scalar step size to be chosen from a specified range. More often than not the subgradient γ^r is computed by the relationship $\gamma^r = A\bar{x}^r - b$. Some theoretical results relating to an appropriate step size θ_r are available. The results presented by Held, Wolfe and Crowder (1974) are presented here. The general result is that $\omega(\lambda^r)$ will converge to its minimum ω^* under merely the conditions: $\theta_r \rightarrow 0$, $\sum_{r=0}^{\infty} \theta_r = \infty$. Alternatively, using the choice $\theta_r = \rho_r (\omega(\lambda^r) - \hat{\omega}) / |\gamma^r|^2$ with $\hat{\omega} > \min \omega(\lambda)$, and for each r , $0 < \rho_r \leq 2$, the sequence $\omega(\lambda^r)$ either converges to $\hat{\omega}$ or a vector λ^r is obtained such that $\omega(\lambda^r) \leq \hat{\omega}$.

If λ^* is the optimal vector, and if the step size θ_r is sufficiently small, the vector λ^{r+1} would be closer to λ^* than λ^r is. Thus the sequence approaches the optimal vector, although the objective values $\omega(\lambda^r)$ are not monotonic. There seem to be very few theoretical results available on the rate of convergence of the subgradient optimization algorithm. Some of the studies have reported that proper estimation of $\hat{\omega}$ plays a role in the rate of convergence. Held et al. (1974) have reported that the outcome does not depend critically on the correct value of the estimated $\hat{\omega}$. However, it is necessary that $\theta_r \rightarrow 0$, but the alternative choice of θ_r using the constant ρ_r does not accomplish this, unless a sequence of ρ_r is selected, which tends towards zero. One of the suggested sequences is to set $\rho = 2$ for $2n$ iterations (where n is a measure of the problem size), and then successively halve both the value of ρ and the number of iterations until the number of iterations reaches some threshold value Z ; ρ is then halved every Z iterations until the resulting θ_r is sufficiently small. This procedure violates the condition $\sum \theta_r = \infty$. It is thus possible to converge to a nonoptimal value of λ . Another method for the selection of ρ_r is to start with $\rho = 2$, and halving ρ_r whenever $\omega(\lambda^r)$ has failed to move in the desired direction (i.e. either increase or decrease) in some fixed number of iterations.

To sum up, the present practice and general observation on the use of the subgradient optimization algorithm are as follows:

- (i) the subgradients are generally computed by the relationship $\gamma^r = Ax^r - b$; due to ease of computation of these subgradients.
- (ii) the multiplier vector λ is calculated by the relationship $\lambda^{r+1} = \max(\lambda^r + \theta_r \gamma^r, 0)$, the scalar step size θ_r is computed by the relationship $\theta_r = \rho_r (\omega(\lambda^r) - \hat{\omega}) / |\gamma^r|^2$ where $0 < \rho_r \leq 2$, and some suitable method is selected for changing the value of ρ_r .
- (iii) the rate of convergence of the objective function is not monotonic and is very slow at later stages.
- (iv) though in some cases it has been possible to get the optimal solution through the subgradient optimization technique, in most cases it merely gives a very good bound and branch and bound has to be resorted to in order to get the optimal solution.

The inherent simplicity and proper choice of constraints set $Bx \leq d$ has led to the wide application of Lagrangean relaxation, specially in getting a tight bound in the branch and bound procedure. As a result, it is difficult to summarise all the efforts in this direction made so far, especially since 1970 and more so from the later half of 70's. Fisher (1981) has surveyed and summarised a large volume of studies in the area of Lagrangean relaxation. The study by Fisher (1981) contains a large number of known applications of Lagrangean relaxation.

7. Solution Methods for the Timetabling Problems

The problem (IP) is the representation of a general integer linear programming problem. Two relaxation approaches can be adopted for solving (IP): one by relaxing the integer requirements of the variables and the other by relaxing some of the constraints and solving the associated Lagrangean problem. Relaxing some of the

constraints and solving the resultant Lagrangean problem, which is an integer program, gives a better bound than the linear programming relaxation of the original problem (Geoffrion 1974). Hence only Lagrangean relaxation approach has been considered here.

The conventional Lagrangean approach is to form the constraints of the problem into two sets; e.g. (i) $Ax \leq b$ and (ii) $Bx \leq d$. One set of the constraint is dualized by using Lagrange multipliers and added to the objective function. The problem (IPR _{λ}) represents the problem (IP) when one set of constraints is dualised, namely the set $Ax \leq b$. The problem is now solved with the remaining constraint set $Bx \leq d$. The constraint set $Bx \leq d$ is selected in such a way that it can be solved by using some simple heuristics. The constraints of the timetabling problem (P) have been formed into two sets in order to explore the possibility of using the solution method suggested by Geoffrion. The constraint (P-II) of problem (P) is designated to be the set $Ax \leq b$ and the constraints (P-I) and (P-III) are designated to be the set $Bx \leq d$. Now, the Lagrangean relaxation of the problem (P) can be represented as follows:

$$\begin{aligned} \text{maximise} \quad & \sum_{i=1}^{\text{NSG}} \sum_{j=1}^{\text{NP}} c_{ij} x_{ij} + \lambda_{kj} \left(a_{kj} - \sum_{i \in R_k} x_{ij} \right), \quad k = 1, \dots, \text{NRG}, \quad j = 1, \dots, \text{NP}, \\ & \text{(PRH}_{\lambda}) \\ \text{subject to} \quad & \sum_{j=1}^{\text{NP}} x_{ij} = \text{NSPG}_i, \quad i = 1, \dots, \text{NSG}, \\ & \text{(PRH}_{\lambda}\text{-I)} \\ & \sum_{i \in T_l} x_{ij} \leq 1; \quad j = 1, 2, \dots, \text{NP}, \quad l = 1, 2, \dots, \text{NTG}, \quad x_{ij} = 0, 1. \\ & \text{(PRH}_{\lambda}\text{-II)} \end{aligned}$$

The problem (PRH _{λ}) for known values of λ_{kj} is solved by a heuristic algorithm. The heuristic is based upon the special nature of the two sets of constraints. The variable x_{ij} is a binary variable, i.e. it can take only two values, either 0 (zero) or 1 (one). There are NP variables for a given value of 'i' and according to constraint set (PRH _{λ} -I), NSPG _{i} of these NP variables must take the value '1'. Since (PRH _{λ}) is a maximisation problem, it is sensible to assign the value of '1' to the NSPG _{i} variables x_{ij} (out of NP) for a given 'i' which have higher objective function coefficients compared to the remaining variables. According to the constraint set (PRH _{λ} -II), at most one of the variable x_{ij} for a given 'j' and 'i' in $\in T_l$ can take the value of '1'. Further since the subsets T_l ($l = 1, 2, \dots, \text{NTG}$) of the subject groups are not mutually exclusive, a particular variable x_{ij} may occur in more than one constraint in the constraint set (PRH _{λ} -II). Therefore, as soon as a particular variable x_{ij} is assigned the value of '1' all the other variables in the constraints in which x_{ij} appears in (PRH _{λ} -II) must be assigned the value of '0'. The above special nature of the constraint sets (PRH _{λ} -I) and (PRH _{λ} -II) has been incorporated in the heuristic algorithm described below, for the solution of the problem (PRH _{λ}).

The various steps of the heuristic algorithm are as stated below.

(1) The variables are serialised in order of their objective function coefficient values (c-values). The variable with the highest objective function coefficient is the first and the variable with the lowest objective function coefficient is the last. In between these two extremes, the variables are arranged in order of decreasing objective function coefficient. It may be mentioned here that, with different set of multiplier values λ_{kj} , the objective function coefficient for the variables will be different and so the ordering of the variables has to be done again.

(2) One of the binary variables x_{ij} is selected from the serialised order to be assigned the value of '1'.

(3) All the other variables (conflicting variables) which appear along with the selected variables in the constraints (there may be more than one) in $(PRH_{\lambda-II})$ are identified so that these are not selected for assigning the value '1' else the constraint set $(PRH_{\lambda-II})$ will be violated.

(4) Steps 1, 2, and 3 are continued until a constraint of the set $(PRH_{\lambda-I})$ is satisfied for a subject group 'i'. At this stage all the remaining variables in the constraint for subject group 'i' in the set $(PRH_{\lambda-I})$ are identified, so that these are not selected for assigning the value '1' else the constraint set $(PRH_{\lambda-I})$ will be violated.

Steps 1 to 4 are continued until all the variables are considered in the decreasing order of c , or all the constraints of the set $(PRH_{\lambda-I})$ are satisfied.

The solution obtained by the heuristic algorithm is checked for its feasibility with respect to the dualised constraint set $\sum_{i \in R_k} x_{ij} \leq a_{kj}$, $k = 1, 2, \dots, NRG$, $j = 1, 2, \dots, NP$.

If the heuristic solution is feasible with respect to the above constraint set, it is a solution to the timetabling problem (P). If the solution is not feasible with respect to the above constraint set, then the extent of infeasibility in the constraints of the set forms the basis for computation of the multiplier vector λ_{kj} .

A subgradient method similar to one suggested by Held, Wolfe and Crowder (1974), and by Fisher, Northup and Shapiro (1975) has been used for the computation. The method involves computation of the multiplier vector sequentially using the relationship, $\lambda^{r+1} = \max\{\lambda^r + \theta_r \gamma^r, 0\}$; where γ^r is the vector having $[\sum_{i \in R_k} x_{ij} - a_{kj}]$ and its (k, j) th component at the r th iteration. The scalar $\theta_r = \rho_r (\omega(\lambda^r) - \hat{\omega}) / |\gamma^r|^2$ where $\omega(\lambda^r)$ is the objective value of the Lagrangean relaxed problem with multipliers λ^r . In case of the problem (PRH_{λ}) , it is the objective value with the multipliers λ'_{kj} . $\hat{\omega}$ is the aimed at objective value. In a problem with all c_{is} 's as zeros, $\hat{\omega} = 0$.

ρ_r is a constant term, such that $0 < \rho_r \leq 2$. In this case it has been taken to be 2. In most published cases the value of ' ρ_r ' is progressively reduced after a certain number of iterations.

In case of (PRH_{λ}) , the starting value of λ_{kj} is a '0' vector and then the λ 's are calculated by the above relationship. The solution procedure iterates between the heuristic algorithm steps 1 to 4, and the computation of multipliers λ_{kj} . The procedure is terminated when a solution generated by the heuristic algorithm is found to be feasible in the dualised constraint set. Alternatively the procedure is terminated after performing a specified number of iterations. The algorithm combining the heuristic steps and the subgradient method has been tested for a number of test problems. The results of these test runs are as follows:

(1) The method has worked well for a number of moderately constrained (with respect to the dualised constraints) small and medium sized problems, e.g. a problem with 20 subject groups, two room groups, 11 student groups and 35 periods per week could be solved in 34 seconds on the CDC 7600 with less than 150 iterations. But the method failed in some other problems. In particular, in a case of a tightly constrained test problem with known solution, the algorithm failed to get a solution in 200 iterations.

(2) There is no monotonic increase in the objective value $\omega(\lambda^r)$ from iteration to iteration.

(3) In the later stages, there can be a cyclic variation in the objective value over a certain number of iterations.

(4) There is also cyclic variation in the amount of infeasibility associated with the dualised constant, in the later stages.

Various modifications to the heuristic cum subgradient algorithm, which have been tried out to overcome the drawbacks, are mentioned below with the outcome associated with them.

(a) The value of the constant term ' ρ ' has been reduced from '2' in stages after a certain number of iterations. This modification has not given encouraging results in terms of reducing the infeasibility in the dualised constraints.

(b) The method for the computation of multipliers ' λ ' has been changed to: $\lambda^{r+1} = \max\{\lambda^r + \theta_r \gamma^r, \lambda^r\}$, i.e. the multipliers are calculated in a way such that they never decrease. This method has been encouraging, insofar as it reduced the infeasibility in the dualised constraints, but it has not resulted in getting a solution of the problem (PRH _{λ}).

(c) A procedure to assign the value of '0' to some variables after every 10 iterations, such that the dualised constraints are not violated, has also been of little effect in overcoming the drawbacks.

It was evident from these observations that branching has to be resorted to to get a solution in each case. This has not been incorporated in the algorithm, as it was understood from the behavior of the heuristic algorithm that it may fail to produce a feasible solution in a tightly constrained problem. However, a suitably designed heuristic is potentially preferable especially in view of less solution time in moderately constrained timetabling problems.

8. Relaxed Problem with Unimodularity

The subsequent attempt to solve the problem (P) is based on a different division of the set of constraints into two groups. The purpose of using Lagrangean relaxation is to have a relaxed problem which is simpler to solve than the original problem itself such that computational advantages can be realised. Preferably the relaxed problem should possess considerable special structure, so that an efficient specialised algorithm or simple heuristic can be used. The earlier division was done in such a way that a simple heuristic could be used, but it had its limitation as discussed. The subsequent division is based on exploiting the special structure of the problem. The problem formulated for the purpose is as follows:

$$\text{maximise} \quad \sum_{i=1}^{\text{NSG}} \sum_{j=1}^{\text{NP}} c_{ij} x_{ij} + \pi_{lj} \left(1 - \sum_{i \in T_l} x_{ij} \right), \quad l = 1, 2, \dots, \text{NTG}, \quad j = 1, 2, \dots, \text{NP}, \quad (\text{PR}_{\pi})$$

$$\text{subject to} \quad \sum_{j=1}^{\text{NP}} x_{ij} = \text{NSPG}_i; \quad i = 1, 2, \dots, \text{NSG}, \quad (\text{PR}_{\pi}\text{-I})$$

$$\sum_{i \in R_k} x_{ij} \leq a_{kj}; \quad j = 1, 2, \dots, \text{NP}, \quad k = 1, 2, \dots, \text{NRG}, \quad (\text{PR}_{\pi}\text{-II})$$

$$x_{ij} = 0, 1.$$

The problem (PR _{π}) is formulated by dualising the constraint set (P-III) of problem (P) and incorporating it in the objective function with multipliers π_{lj} . The relaxed problems (PR _{π}) are unimodular. This is so, as x_{ij} occurs only once in (PR _{π} -I) and also only once in (PR _{π} -II) due to the set R_k being exhaustive and mutually exclusive. Due to unimodularity, the problem (PR _{π}) has the integrality property. That is even if the integrality constraints on the variables x_{ij} are relaxed to $0 \leq x_{ij} \leq 1$ there will still be an integral optimal solution. The integer linear program (PR _{π}), because of its special structure, can be solved optimally for a given set of values for π_{lj} , by using relatively simple techniques. Various alternatives considered for the purpose are described below.

(a) The use of linear programming method for solving (PR _{π}) has been considered. The linear programming solution of (PR _{π}) with the integer requirements of variables x_{ij} relaxed to linearity $0 \leq x_{ij} \leq 1$ will result in integer solution of x_{ij} due to the integrality

property. The likely disadvantages of using this method are:

(i) the linear programming algorithm requires a sizeable amount of storage for storing problem data and other information for computational purposes. The timetabling problem considered here is big and the large storage requirement by this method is less likely to be a step in the right direction.

(ii) the problem (PR_{π}) has to be solved optimally, by using the linear programming algorithm every time the values of π_{ij} are changed. This will result in considerable computation time, which is undesirable for a problem with special structure. Because of these disadvantages, the use of a general algorithm like linear programming to solve a problem with favourable special structure has not been explored further.

(b) The application of transportation algorithm, which takes advantage of the unimodularity, to solve (PR_{π}) has been thought to be a possibility. A standard 'Transportation Algorithm,' which is quite simple and fast, cannot be used in this case. Here the variables are bounded and so a bounded transportation algorithm needs to be used. In (PR_{π}) all the variables are bounded and the permissible range is very narrow (between zero and one). As a result a number of variables will be nonbasic at their upper bound, and hence with this nature, the transportation algorithm with necessary modifications to take care of bounded variables is less likely to be efficient in solving (PR_{π}) .

(c) On the basis of (a) and (b), the algorithm which can be considered suitable must take advantage of the unimodularity and should be able to take care effectively of the bounded nature of all the variables. It may be noted that the problem (P) does not have the integrality property. It is only problem (PR_{π}) which has the integrality property. Due to this integrality property, the bound obtained on (PR_{π}) will not be better than that obtained by using LP (Geoffrion 1974). The use of the proper algorithm for solving the relaxed problem (PR_{π}) will result in computational advantages only. Next, an 'Out of Kilter' algorithm has been used towards this end.

9. Out-of-Kilter Method

The 'Out-of-Kilter' codes available in MPCODE at the U.L.C.C. (University of London Computer Centre), where the study has been carried out, is based on a published algorithm by Bray and Witzgall (1968). The Out-of-Kilter solves an LP problem with the following characteristics:

$$\text{minimise } cx, \quad \text{subject to } Ax = 0, \quad 1 \leq x \leq u,$$

where A is the incidence matrix of a directed graph. Each variable x is an arc with a lower and upper limit on the flow through this arc. The incidence matrix A by definition is unimodular. In addition a bound on each variable is also incorporated in the problem. Therefore, the Out-of-Kilter algorithm takes care of the unimodularity of the problem (PR_{π}) and also the upper bound on each variable x_{ij} which is '1' (one). The problem (PR_{π}) is not in the form specified and new variables have to be defined to present it in the above form to be solved by the Out-of-Kilter algorithm. After simplification of the objective function and omitting the constant term in the objective function, the problem (PR_{π}) can be represented as follows:

$$\begin{aligned} \text{maximise } & \sum_{i=1}^{NSG} \sum_{j=1}^{NP} c_{ij} x_{ij}, \quad \text{subject to} \\ & \sum_{j=1}^{NP} x_{ij} = NSPG_i; \quad i = 1, 2, \dots, NSG \\ & \sum_{i \in R_k} x_{ij} \leq a_{kj}; \quad j = 1, 2, \dots, NP, \quad k = 1, 2, \dots, NRG, \quad x_{ij} = 0, 1. \end{aligned}$$

After adding new variables the problem can be presented in the following form:

minimise

$$\sum_{i=1}^{NSG} \sum_{j=1}^{NP} -\bar{c}_{ij} x_{ij},$$

subject to

$$\sum_{j=1}^{NP} -x_{ij} + \theta_i = 0; \quad i = 1, 2, \dots, NSG,$$

$$\sum_{i \in R_k} x_{ij} - \beta_{kj} = 0; \quad k = 1, 2, \dots, NRG, \quad j = 1, 2, \dots, NP,$$

$$\sum_{i=1}^{NSG} -\theta_i + \sum_{k=1}^{NRG} \sum_{j=1}^{NP} \beta_{kj} = 0,$$

Variable	θ_i	β_{kj}	x_{ij}
lower bound	$NSPG_i$	0	0
upper bound	$NSPG_i$	a_{kj}	1

(PR_π) has now been represented in the form required by the Out-of-Kilter algorithm. Figure 1 is the network representation of the problem. Each variable appears as an arc in the diagram and each constraint appears as a node in the diagram, including the added constraint, the dummy node.

The constraints of the problem (PR_π) has been presented in the form $Ax = 0$, by adding notional arcs and vertices. The constraints $Ax = 0$ imply zero net flow at every vertex. During the solution process the algorithm maintains the condition $Ax = 0$ throughout and proceeds to satisfy the remaining primal and dual conditions, one column (variable, arc) at a time. Once an arc has been put ‘in Kilter,’ it is not allowed to go ‘Out-of-Kilter.’

The algorithm starts with zero flows (primal solution) and zero prices (dual solution). Starting with all the flows as zeros satisfies the condition $Ax = 0$ and this condition is never violated. With this starting condition, the arcs which are Out-of-Kilter are noted and one of them at a time is considered for bringing them ‘in-to-Kilter.’ While doing so, the arcs which are in Kilter are never made Out-of-Kilter. The method follows a labelling procedure for the nodes to change the flows through certain arcs accompanied with the changes in dual values.

The selection of the ‘Out-of-Kilter’ algorithm to solve the relaxed problem (PR_π) is due to its ready availability and simplicity. The algorithm takes advantage of uni-

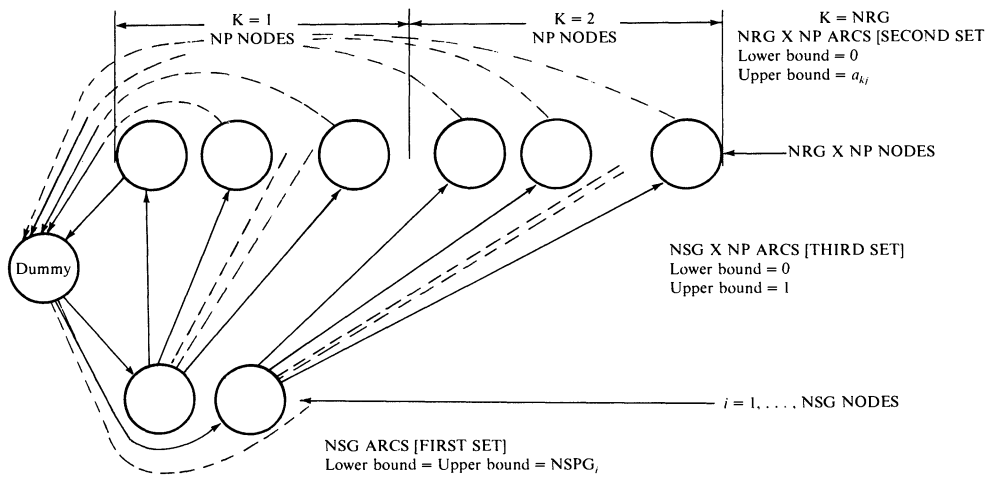


FIGURE 1

modularity of the problem (PR_π) and upper bounding on all the variables. Another aspect of the algorithm is its modest storage requirement, which is little over 6 times the number of arcs in the problem, making it suitable for application to large size problems. The Out-of-Kilter algorithm solves optimally the problem (PR_π) for a given set of values for the vector π . However, (PR_π) has to be optimized over nonnegative values of π , and so the value of the multiplier vector has to be computed.

The computation of the multiplier vector π can be considered to be the penalty associated with the constraints in (P-III) which are not satisfied by the solution of (PR $_{\pi}$). The vector π modifies the objective function for (PR $_{\pi}$), by penalising the variables in the unsatisfied constraints of (P-III). This dictates the solution of the Out-of-Kilter algorithm, as in this way feasibility is forced on the constraint set (P-III), thereby achieving a solution of (P). However, it has been observed that, though this method improves the feasibility on the constraint set (P-III), it fails to get complete feasibility and in fact the method does not improve the feasibility (in terms of reducing infeasibility) after a certain number of iterations of the Out-of-Kilter and the sub-gradient optimization algorithms. Hence, branch and bound has been incorporated into the solution process after a certain number of predetermined iterations. A flow chart of the solution process is presented in Figure 2. The Out-of-Kilter algorithm used to solve the problem (PR $_{\pi}$) has been represented by the block Out-of-Kilter. Other subroutines are enclosed by dotted lines in the flow chart.

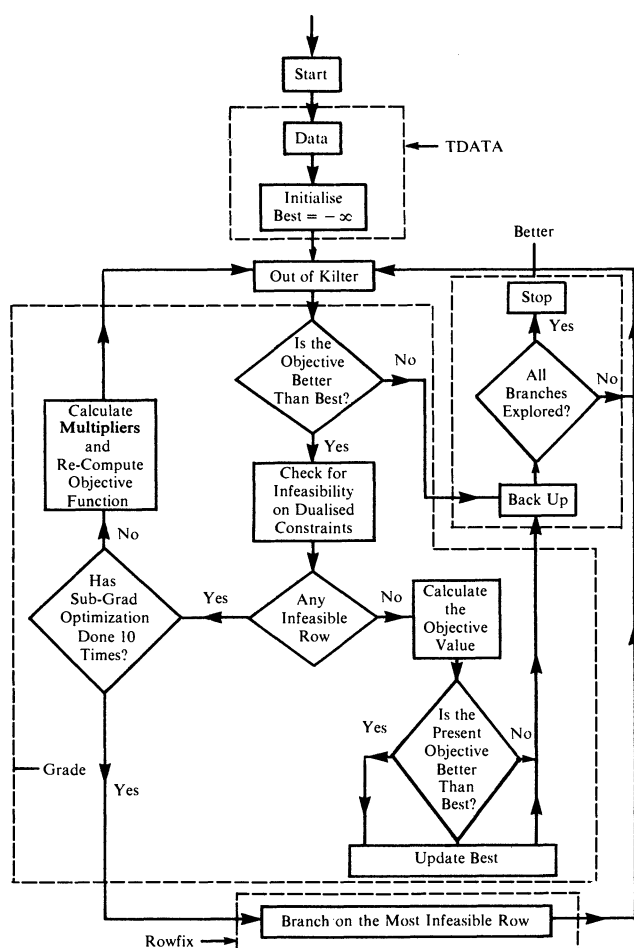


FIGURE 2. Flow Diagram of the Algorithm. The --- Lines Enclose Various Subroutines.

It may be noted that all the variables are bounded in (PR_{π}). Due to this the problem is highly degenerate in nature, which can cause computational problems with a simplex based algorithm like transportation. The Out-of-Kilter algorithm is not simplex based and as such is a better algorithm for this problem.

10. Branch and Bound Procedure

A branch and bound method was developed by Land and Doig (1960) to solve discrete programming problems. Later the method was modified by Dakin (1965). The branch and bound procedure used here is an extension of the same principle.

It has been observed that after about 10 iterations of the subgradient method the improvement in feasibility of the dualised constraints is very little. Based on this observation, the branch and bound procedure is adopted after 10 iterations of the subgradient method. The algorithm identifies the most infeasible constraint (row) in the dualised constraint set in every iteration of the subgradient method. It has been observed that on some occasions the most infeasible row (INFR) is the same in two consecutive iterations of the subgradient algorithm. This suggests that the subgradient method has failed to improve the feasibility of INFR. In these circumstances the branch and bound procedure is adopted immediately without any regard to whether 10 iterations of the subgradient method have been performed.

The most infeasible constraint INFR is selected for branching. The variables having nonzero coefficient in this row can be considered to belong to a set similar to a 'special ordered set' (Beale and Forest 1976), as only one of the variables can take the value of 1.0 and all other variables must take the value of 0.0, in order not to violate the feasibility in this row. The dualised constraints, to which INFR belongs, are the constraint set (P-III) of the problem (P). The nonzero coefficients in the constraint set (P-III) are 1's and the right-hand sides of the constraint set are 1's. Therefore, the variables being binary, only one of the variables can take the value of '1' and the rest of the variables of the set (having nonzero coefficients) must take the value of '0' (zero).

The branching strategies, those which could be considered, are one or more of the following.

(i) As the problem is one of maximising, the variable with the highest objective function value should be chosen for being fixed at a value of 1.0.

(ii) In the timetabling problem we are basically concerned with a feasible solution. Therefore a variable which has least number of periods available for it to be scheduled should be chosen for being assigned the value of 1.0.

(iii) It may as well be said, from feasibility considerations, that a variable which has maximum number of conflicting variables is most difficult to schedule, and so it should be assigned the value of 1.0.

Some trials have been carried out to find out the effectiveness of the strategies (ii) and (iii). It has been found that strategy (ii) is more effective in finding a solution quickly compared to strategy (iii). Accordingly, the present branching strategy is based on the combination of strategy (i) and strategies (ii). However, during the back up, a branch with another variable fixed at 1.0 is explored, provided it results in a better bound compared to the best solution found so far. At a particular level of branching a variable fixed at 1.0 appears in one of the constraints of the set (P-I). One more set of variables of special type can be identified with this constraint. In this case it is similar to a Generalized Upper Bounding Constraint. This fact is made use of to fix some more variables at a value of 0.0. Further, the variable fixed at 1.0 may also appear in one or more of the constraints of the set (P-III) (the dualised constraint set), according to whether other student groups are expected to attend the subject group correspond-

ing to the variable fixed at 1.0 (this is because subsets T_1 ; $1 = 1, 2, \dots, NTG$; are not mutually exclusive). In such a case all the other variables of these constraints are fixed at 0.0. In this way, at a particular level of branching, one variable is fixed at 1.0, and many variables are fixed at 0.0.

11. Computational Procedures and Results

The size of the problem to be solved with the help of the solution method discussed is very large. Two difficulties are likely to be encountered during computation while solving such large problems, (i) large storage requirement, and (ii) long computational time. In order to take care of large storage requirement, the variables are grouped in common blocks. They are incorporated in various subroutines through CDC UPDATE call statements, viz. *CALL (common block name). By adopting this method only the required block is kept in the main memory during computation and there is no necessity to keep all the variables stored in the main memory. Due to long computational time, it may not be possible (or desirable) to complete the job in one computer run. Therefore, a rerunning facility has been incorporated. The time in seconds after which a run has to be terminated is read in the input data cards. After the elapse of the specified computational time, the values of the variables at that stage are written onto a file. In the next run the data from the file are retrieved and the computation is continued from the stage at which the last run was terminated.

The timetabling problem, the computational result of which is presented below, involves scheduling of both double period and single period classes. The problem has been solved in two stages. First, the double period classes are scheduled and then the remaining periods are used for scheduling single period classes.

	Double Period Timetabling	Single Period Timetabling
Numbers of periods per week	18	36
Number of nodes in the out-of-kilter problem	185	275
Number of arcs in the out-of-kilter problem	1876	3658
Number of dualised constraints	594	1188
Total number of variables in the problem (in terms of problem (P))	1692	3384
Number of variables to take nonzero values	155	174
Number of nodes in the branch and bound tree	36	34
Total number of variables fixed (either at zero or at one) prior to getting the first solution	687	988
Total CPU time including input and output of the reruns (in CDC 7600) sec	228	473
The other details of the problem are as follows:		
Number of student groups:	33	
Number of subject groups:	94	
Number of room groups:	5	

The computations were carried out in CDC 7600, Cyber 72, under scope 2.1.4. The CPU time includes input, output for all the returns and initial data input and compilation of the program. The initial data entry was through punched cards. The

output at the end of each run was written on to disc file which formed the input for the following run. The number of runs were five for the double period timetabling and seven for the single period timetabling.

The algorithm has also been used for scheduling of optional subjects (Tripathy 1980) with improved result compared to an heuristic. The algorithm has also been tried with some examination scheduling problem.

12. Conclusion

The present study has explored the possibility of applying mathematical programming techniques for the solution of timetabling problems. Proper formulation and suitable partitioning of the timetabling problem has made it possible to use the Lagrangean relaxation technique effectively. The study has reaffirmed the potential of Lagrangean relaxation for solving large-scale integer linear programming problems. The solution method proposed here can be used with success not only for many similar timetabling problems but also can be extended to various other similar problems and examination timetabling is one such problem commonly encountered.¹

¹The author gratefully acknowledges the advice and guidance of Professor A. H. Land of the London School of Economics, throughout the above study and the support of the Science Research Council of the United Kingdom. The author is also grateful to the referees for their constructive comments.

References

- AKKOYUNLU, E. A., "A Linear Algorithm for Computing the Optimum University Timetable," *Computer J.*, 16, 4 (1973), 347–350.
- ALMOND, MARY, "An Algorithm for Constructing University Time-Table," *Computer J.*, 8 (1966), 331–340.
- , "A University Faculty Timetable," *Computer J.*, 12 (1969), 215–217.
- ANTHONISSE, J. M. AND B. J. LAGEWEG, "Timetables for Dentistry Students at the University of Amsterdam," Third European Congress on Operations Research, Amsterdam, 9–11 April 1979.
- APPLEBY, J. S., D. V. BLACK, AND E. A. NEWMAN, "Techniques for Producing School Timetables on a Computer and their Application to Other Scheduling Problems," *Computer J.*, 3 (1961), 237–245.
- AUST, R. J., "The School Timetabling Problem," Ph.D. Thesis, School of Applied Science, University of Sussex, 1973.
- , "An Improvement Algorithm for School Timetabling," *Computer J.*, 19 (1976), 339–345.
- BARRACLOUGH, E. D., "The Application of a Digital Computer to the Construction of Timetables," *Computer J.*, 8 (1965), 136–146.
- BEALE, E. M. L. AND J. J. H. FOREST, "Global Optimization Using Special Ordered Sets," *Mathematical Prog.* 10 (1976), 52–69.
- BRAY, T. A. AND C. WITZGALL, "Algorithm 336 'NETFLOW (H)'," *Comm. ACM* 11, 9 (1968).
- BRITTON, J. N. G. AND F. J. M. FARLEY, "College Timetable Construction by Computer," *Computer J.*, 14 (1971), 361–365.
- CSIMA, J., "Investigations on a Timetable Problem," Doctoral Dissertation, University of Toronto, 1965.
- DAKIN, R. J., "A Tree Search Algorithm for Mixed Integer Programming Problem," *Computer J.*, 8, 3 (1965), 250–255.
- DEMPSTER, M. A. H., "On the Gotlieb–Csimá Timetabling Algorithm," *Canad. J. Math.*, 20 (1968), 103–119.
- , "Two Algorithms for the Timetabling Problem," in *Combinatorial Mathematics and Its Application*, D. J. A. Welsh, (ed.) Academic Press, New York, 1971, 63–85.
- DEWERRA, D., "Construction of School Timetable by Flow Method," *INFOR*, 9 (1971), 12–22.
- , "Notes—Some Comments on a Note about Timetabling," *INFOR*, 16 (1978), 90–92.
- DUTT ROY, A., "School Timetabling via Integer Programming—A Test Bed for an Integer Programming Package," M.Sc. Thesis, Institute of Computer Science, London University, 1972.
- FISHER, M. L., "Lagrangean Relaxation Methods for Solving Integer Programming Problems," *Management Sci.*, 27, 1 (1981), 1–18.
- , W. D. NORTHUP, AND J. F. SHAPIRO, "Using Duality to Solve Discrete Optimization Problems: Theory and Computational Experience," *Math. Prog. Study*, 3 (1975), 56–94.
- GEOFFRION, A. M., "Duality in Nonlinear Programming: A Simplified Applications-Oriented Development," *SIAM Rev.*, 13, 1 (1971), 1–37.
- , "Lagrangean Relaxation for Integer Programming," *Math. Prog. Study*, 2 (1974), 82–114.
- GOTLIEB, C. C., "The Construction of Class-Teacher Timetable," Proceedings of I.F.I.P. Congress, Munich (Amsterdam), 1962, 73–77.

- GRUMAN, A. J., "A Comparison of Computer Programs for the Assignment of Students to Classes: Report of an Experimental Study," *J. Educational Data Processing*, 4 (1966), 7–15.
- HELD, M. AND R. M. KARP, "The Travelling Salesman Problem and Minimum Spanning Trees," *Oper. Res.*, 18 (1970), 1138–1162.
- AND ———, "The Travelling Salesman Problem and Minimum Spanning Trees: Part II," *Math. Prog.*, 1 (1971), 6–25.
- , P. WOLFE, AND H. P. CROWDER, "Validation of Subgradient Optimization," *Math. Prog.*, 6 (1974), 62–88.
- JUNGINGER, W., "Reducing the Timetable Problem to a Three Dimensional Transportation Problem," *Z. für Oper. Res.*, 16 (1972), 11–25.
- LAND, A. H. AND A. G. DOIG, "An Automatic Method of Solving Discrete Programming Problem," *Econometrica*, 28 (1960), 497–520.
- LAPORTE, G. AND S. DESROCHES, "Examination Timetabling by Computer," Working Paper No: G-82-03, Ecole des Hautes Etudes Commerciales de Montreal, 1982.
- LAWRIE, N. L., "An Integer Linear Programming Model of a School Timetabling Problem," *Computer J.*, 12 (1969), 307–316.
- AND H. VEITCH, *Timetabling and Organisation in Secondary Schools*, NFER Publishing Company, England, 1975.
- LIONS, J., "Matrix Reduction Using the Hungarian Method for the Generation of School Timetable," *Comm. ACM*, 9 (1966), 349–354.
- , "The Ontario School Scheduling Program," *Computer J.*, 10 (1967), 14–21.
- Local Government Operational Research Unit, "The Use of Computers for School Timetabling," Report No. C19.
- MCDIARMID, C. J. H., "The Solution of a Timetabling Problem," *J. Inst. Math. Appl.*, 9 (1972), 23–34.
- MURPHY, J. AND R. SUTTER, "School Scheduling by Computer—The Story of GASP," Educational Facilities Labs., Inc. New York, December 1964.
- OAKFORD, ROBERT V., DWIGHT W. ALLEN, AND LYNNE A. CHATTERTON, "School Scheduling Practice and Theory," *J. Educational Data Processing*, 4 (1966), 16–50.
- PUNTER, ALAN, "School Timetabling by Computer. I—A Graph Colouring Formulation," unpublished working paper, 1977a.
- , "School Timetabling by Computer. II—A System Based on Graph Colouring," unpublished working paper, 1977b.
- SHARRAT, M., "Timetabling at the Management Centre," M.Sc. Thesis, Bradford University, (1970).
- TRIPATHY, A., "A Lagrangean Relaxation Approach to Course Timetabling," *J. Oper. Res. Soc.*, 31 (1980), 599–603.
- ULPH, A., "Notes—An Extension of a Result in School Timetabling," *INFOR*, 15 (1977), 255–257.
- VAN DUSSELDORP, RALPH A. AND D. E. RICHARDSON, "Class Scheduling by Computer," *J. Educational Data Processing*, 4 (1966), 51–60.
- WELSH, D. J. A. AND M. B. POWELL, "An Upper Bound for the Chromatic Number of a Graph and Its Application to Time-Tabling Problem," *Computer J.*, 10 (1967), 85–86.
- WHITE, D. J., "A Note on Faculty Timetabling," *Oper. Res. Quart.*, 26 (1976), 875–878.
- WOOD, D. C., "A System for Computing University Examination Timetables," *Computer J.*, 11 (1968), 41–47.
- , "A Technique for Colouring a Graph Applicable to Large Scale Timetabling Problems," *Computer J.*, 12 (1969), 317–319.
- YULE, A. P., "Extensions to the Heuristic Algorithm for University Timetables," *Computer J.*, 10 (1967), 360–364.